

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт о выполнении курсовой работы

По дисциплине: «Теоретические основы баз данных»

На тему: «Учёт компьютерных шрифтов электронной типографии»

№ этапа	Дата	Подпись
1		
2		
3		

Выполнил студент
группы 5130201/40002

_____ Семенов И. А.

Проверил
преподаватель

Попов С. Г.

«_____» 2026г.

Санкт-Петербург
2026

Содержание

1	Аналитика	3
1.1	Описание предметной области «Учёт компьютерных шрифтов в электронной типографии»	3
1.2	Цели функционирования базы данных	6
1.3	Польза от внедрения базы данных	7
1.4	Описание сущностей	7
1.5	Описание ролей и субъектов	8
1.6	ER-диаграмма	9
1.7	Схема объектов	11
2	Проектирование	12
2.1	Лингвистическое описание схемы базы данных	12
2.2	Описание таблиц базы данных	13
2.3	Создание таблиц базы данных и генерация записей	21
Приложение А. Исходный код QSL-скрипта для создания таблиц базы данных		23
Приложение Б. Программа на TypeScript для заполнения базы данных тестовыми данными		25

1 Аналитика

1.1 Описание предметной области «Учёт компьютерных шрифтов в электронной типографии»

Типографика — искусство оформления текста с помощью наборного шрифта — является одним из фундаментальных элементов визуальной коммуникации — передачи информации посредством зрительно воспринимаемых образов. На протяжении столетий она развивалась от ручного набора металлических литер (брусков с выпуклым изображением знака) до современных цифровых технологий. Изобретение печатного станка Иоганном Гутенбергом в середине пятнадцатого века положило начало массовому распространению печатного слова, однако процесс набора текста оставался трудоёмким ремеслом. Каждая литера отливалась из металла, а наборщик вручную составлял строки из отдельных знаков. Появление линотипа (машина, отливающая целые строки) и монотипа (машина, отливающая отдельные литеры) в конце девятнадцатого века механизировало этот процесс, но подлинная революция произошла с переходом к цифровым технологиям.

Электронная типография — организация, осуществляющая подготовку и выпуск печатной и цифровой продукции с использованием компьютерных технологий, — представляет собой современный этап развития печатного дела, в котором все процессы подготовки и воспроизведения текста осуществляются с использованием компьютерных технологий. В отличие от традиционной типографии, где шрифт существовал как физический объект, в электронной типографии шрифт является программным продуктом. Компьютерный шрифт — это файл, содержащий векторные или растровые описания графем (букв, цифр и других знаков письма), а также метрическую информацию, необходимую для корректного отображения и печати текста. Такие шрифты используются в издательских системах, графических редакторах, веб-страницах и пользовательских интерфейсах операционных систем.

В наши дни современную электронную типографию невозможно представить без обширных коллекций шрифтов. Дизайнеры, верстальщики и препресс-специалисты (специалисты по допечатной подготовке) ежедневно работают с десятками гарнитур, подбирая оптимальные решения для каждого проекта. Шрифтовая гарнитура объединяет набор начертаний, выполненных в едином стилистическом ключе. Начертания различаются насыщенностью штрихов (толщиной линий знака) — от сверхтонкого до сверхжирного — и наклоном знаков, формирующим прямое, курсивное (с изменённым рисунком букв) или наклонное (линейно наклонённое прямое) написание. Одна гарнитура может насчитывать от единственного начертания до нескольких десятков вариантов, что позволяет

выделять структурные элементы документа — заголовки, подзаголовки, основной текст.

Шрифты классифицируются по конструктивным особенностям строения знаков. Антиквенные шрифты, также называемые серифными (*от serif*), имеют характерные засечки на концах штрихов. Гротески, или рубленые шрифты (*sans serif fonts*), лишены засечек и отличаются геометрической простотой форм. Моноширинные шрифты (*monospace fonts*) характеризуются одинаковой шириной всех знаков, что делает их незаменимыми в программировании и табличной вёрстке. Акцидентные шрифты предназначены для заголовков, логотипов и декоративных целей, тогда как рукописные имитируют различные виды ручного письма. Шрифты также характеризуются стилистическими особенностями: геометрические построены на основе простых форм, гуманистические отличаются плавными формами, имитирующими ручное письмо пером, брусковые отличаются прямоугольными засечками и одинаковыми толщинами со штрихами.

Важной характеристикой компьютерного шрифта является поддержка языков и письменностей. Глифовый (*от glyph, графическое представление символа*) состав определяет, какие алфавиты способен отображать шрифт. Базовая латиница охватывает западноевропейские языки, расширенная латиница добавляет знаки для центральноевропейских и балтийских языков, кириллица обеспечивает поддержку славянских языков. Системы письма различаются также направлением чтения: латиница и кириллица читаются слева направо, арабское и еврейское письмо — справа налево, традиционные восточноазиатские тексты могут располагаться вертикально сверху вниз. Направление письма влияет на технические требования к отображению шрифта и должно учитываться при организации шрифтовой коллекции.

Компьютерные шрифты существуют в различных технических форматах. Формат TrueType, разработанный компанией Apple в конце восьмидесятых годов, получил широкое распространение благодаря поддержке со стороны операционной системы Windows. Формат OpenType, созданный совместными усилиями Microsoft и Adobe, расширил возможности шрифтов за счёт поддержки дополнительных типографических функций и большего количества глифов. Веб-шрифты в форматах WOFF и WOFF2 оптимизированы для передачи по сети и используются на веб-страницах.

Отображение шрифта в цифровой среде требует постоянного пересчёта. При изменении размера кегля, ширины текстового блока или масштаба страницы программа заново вычисляет положение каждого символа, переносы слов, межстрочные и межбуквенные интервалы. Этот процесс происходит в текстовых редакторах, издательских системах и веб-браузерах. Чтобы сделать пересёт гибче, существуют вариативные

шрифты (*variable fonts*). Они расширяют возможности динамической адаптации, позволяя плавно изменять насыщенность, ширину и другие параметры начертания без загрузки отдельных файлов.

Электронная типография предполагает наличие парка рабочих станций, на которых выполняются задачи вёрстки, дизайна и допечатной подготовки. Рабочая станция представляет собой персональный компьютер с определёнными аппаратными характеристиками: процессором, объёмом оперативной памяти, типом и ёмкостью накопителя, параметрами графического адаптера. На каждой станции установлена операционная система — Windows, macOS или Linux — определённой версии. Сочетание аппаратных характеристик и операционной системы определяет, какие форматы шрифтов поддерживаются и насколько корректно они отображаются.

Шрифты в электронной типографии — лицензируемые продукты. Лицензия определяет допустимые способы использования: некоторые шрифты распространяются бесплатно для любых целей, другие требуют приобретения коммерческой лицензии, третьи доступны по подписке. Лицензионные условия могут ограничивать количество рабочих станций, на которых разрешена установка шрифта, тираж печатной продукции или число просмотров веб-страниц. Без учёта лицензий типография рискует нарушить авторские права.

В электронной типографии работают несколько категорий специалистов. Системный администратор отвечает за техническое состояние парка рабочих станций, установку операционных систем, настройку программного обеспечения и распределение шрифтовых ресурсов между станциями. Дизайнер осуществляет подбор шрифтов для проектов, формирует запросы на приобретение новых гарнитур и использует установленные шрифты в своей работе. Менеджер по закупкам ведёт учёт лицензий, оформляет приобретение новых шрифтов, контролирует сроки действия подписок и соответствие количества установок лицензионным ограничениям.

Процесс пополнения шрифтовой коллекции начинается с выявления потребности. Дизайнер, работая над проектом, может обнаружить необходимость в гарнитуре, отсутствующей в текущей коллекции типографии. Он формирует запрос, указывая название шрифта, требуемые начертания и обоснование выбора. Менеджер по закупкам проверяет наличие шрифта у издательств, сравнивает условия лицензирования и стоимость. После согласования бюджета оформляется приобретение лицензии. Системный администратор получает файлы шрифта и выполняет установку на рабочие станции, для которых предназначена лицензия.

Допечатная подготовка и печать требуют строгого соответствия шрифтов на всех этапах производства. Верстальщик создаёт макет на своей рабочей станции, используя определённые гарнитуры и начертания. За-

тем файл передаётся на допечатную подготовку, затем на печать. На каждом этапе программа обращается к шрифтам, установленным локально. Если на одной из станций нужный шрифт отсутствует, система подставит замену — похожий шрифт с другими метриками. Это приведёт к смещению строк, изменению интервалов, нарушению переносов и выравнивания. Ошибка может остаться незамеченной до печати тиража, а исправление потребует перевёрстки и дополнительных затрат. Централизованный учёт шрифтов позволяет контролировать идентичность коллекций на всех станциях и предотвращать подобные ситуации ещё на этапе передачи макета.

Процесс инвентаризации шрифтовых ресурсов направлен на поддержание актуальности сведений о коллекции. Системный администратор периодически проводит сверку фактически установленных шрифтов на рабочих станциях с данными учётной системы. Выявляются расхождения: шрифты, установленные без учёта, удалённые шрифты, изменения в конфигурации станций. По результатам инвентаризации учётные данные корректируются. Менеджер по закупкам проверяет соответствие количества установок лицензионным ограничениям и при необходимости приобретает дополнительные лицензии или деактивирует избыточные установки.

Таким образом, учёт компьютерных шрифтов в электронной типографии представляет собой комплексную задачу, охватывающую систематизацию сведений о шрифтовых гарнитурах и их характеристиках, регистрацию парка рабочих станций с их аппаратно-программными конфигурациями, отслеживание лицензионных условий и сроков их действия, а также фиксацию связей между конкретными шрифтами и станциями, на которых они установлены. Организация такого учёта обеспечивает соблюдение лицензионных требований, оперативный поиск необходимых шрифтов и рациональное использование типографических ресурсов.

1.2 Цели функционирования базы данных

1. Хранение сведений о шрифтовых гарнитурах, их начертаниях и характеристиках.
2. Хранение информации о рабочих станциях и их аппаратно-программных конфигурациях.
3. Хранение информации о лицензиях на шрифты, условиях использования и сроках действия.
4. Хранение информации об установленных шрифтах на каждой рабочей станции.

1.3 Польза от внедрения базы данных

Польза от внедрения базы данных напрямую следует из поставленных целей. Единый каталог гарнитур позволяет быстро находить нужный шрифт и проверять его характеристики. Реестр рабочих станций с информацией об установленных шрифтах даёт возможность убедиться, что на всех станциях установлены одни и те же шрифты. Учёт лицензий предотвращает нарушение авторских прав и помогает планировать бюджет на продление подписок. Фиксация связей между шрифтами и станциями упрощает выявление расхождений. В результате типография получает инструмент, обеспечивающий согласованность шрифтовых коллекций, соблюдение лицензионных требований и прозрачность использования типографических ресурсов.

1.4 Описание сущностей

1. Гарнитура — это семейство начертаний шрифта, объединённых общим стилем и предназначенных для совместного использования в типографических работах.

Атрибуты: название, год создания, описание, число начертаний.

2. Начертание — это конкретный вариант гарнитуры, определяемый насыщенностью штрихов и наклоном знаков, хранящийся в отдельном файле шрифта.

Атрибуты: название, насыщенность, наклон, имя файла, размер файла.

3. Категория — это классификационная группа, объединяющая шрифты по конструктивным особенностям строения знаков (например, по наличию засечек или ширине символов).

Атрибуты: название, описание, иконка, число гарнитур.

4. Формат — это технический стандарт хранения шрифтовых данных, определяющий структуру файла, способ описания знаков и совместимость с платформами.

Атрибуты: название, расширение, год появления, описание, поддержка веба.

5. Язык — это система письма с определённым алфавитом и направлением чтения, поддержка которой обеспечивается наличием соответствующих знаков в шрифте *начертании шрифта*.

Атрибуты: название, код ISO, направление письма, тип письменности.

6. Алфавит — это совокупность глифов в конкретном начертании шрифта, обеспечивающая отображение текста на определённом языке.

Атрибуты: число начертаний в системе, число полностью поддерживаемых гарнитур, дата проверки, статус.

7. Лицензия — это документ, устанавливающий права и ограничения на использование шрифта, включая количество рабочих станций и срок действия; на каждую гарнитуру распространяются отдельные лицензии.

Атрибуты: тип, срок действия, максимальное число станций, стоимость, дата приобретения.

8. Издательство — это организация, занимающаяся разработкой, производством и распространением шрифтов, а также предоставлением лицензий на их использование.

Атрибуты: название, страна, сайт, год основания, контактный email.

9. Рабочая станция — это персональный компьютер в составе парка типографии, используемый для выполнения задач вёрстки, дизайна и допечатной подготовки.

Атрибуты: инвентарный номер, процессор, объём памяти, тип накопителя, операционная система.

10. Операционная система — это комплекс программ, управляющий аппаратными ресурсами рабочей станции и обеспечивающий среду для работы прикладного программного обеспечения.

Атрибуты: название, версия, разрядность, год выпуска.

11. Использование — это факт размещения конкретного начертания шрифта на рабочей станции, фиксирующий связь между шрифтовым ресурсом и оборудованием.

Атрибуты: дата установки, дата удаления, путь к файлу, статус, дата последнего использования.

1.5 Описание ролей и субъектов

Системный администратор обслуживает парк рабочих станций. Он устанавливает операционные системы, настраивает программное обеспечение и распределяет шрифты между станциями.

Дизайнер использует шрифты в своей работе. Он подбирает гарнитуры для проектов и формирует запросы на приобретение новых шрифтов.

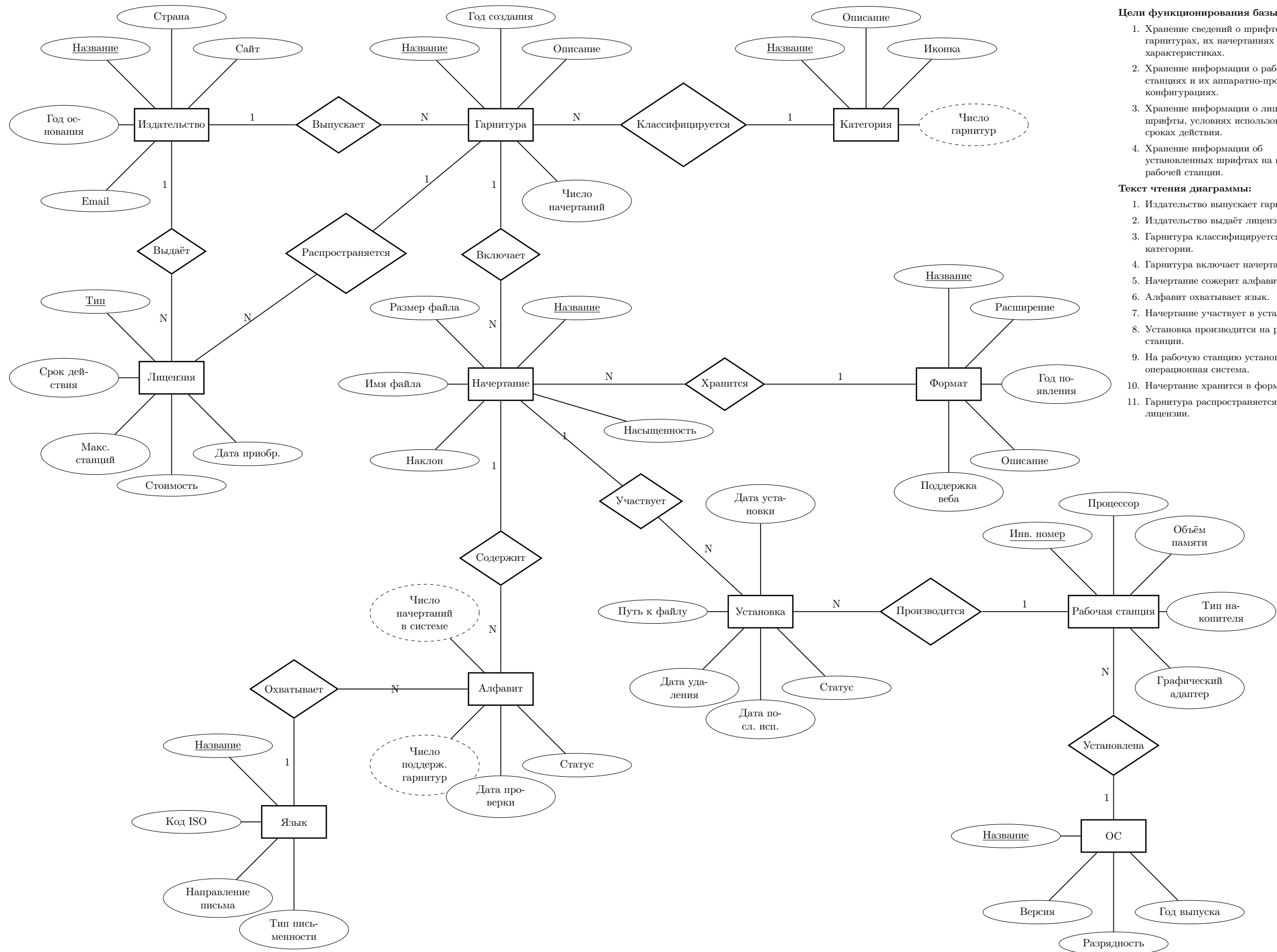
Менеджер по закупкам ведёт учёт лицензий. Он оформляет приобретение шрифтов, контролирует сроки действия подписок и соответствие количества установок лицензионным ограничениям.

Издательство выпускает шрифты и предоставляет лицензии на их использование. Типография приобретает лицензии у издательств и устанавливает шрифты на рабочие станции.

1.6 ER-диаграмма

На рисунке 1 (стр. 10) представлена ER-диаграмма предметной области. Перечень связей:

1. Издательство выпускает гарнитуру.
2. Издательство выдаёт лицензию.
3. Гарнитура классифицируется по категории.
4. Гарнитура включает начертание.
5. Начертание сожержит алфавит.
6. Алфавит охватывает язык.
7. Начертание участвует в установке.
8. Установка производится на рабочей станции.
9. На рабочую станцию установлена операционная система.
10. Начертание хранится в формате.
11. Гарнитура распространяется по лицензии.



Цели функционирования базы данных:

1. Хранение сведений о шрифтовых гарнитурах, их начертаниях и характеристиках.
2. Хранение информации о рабочих станциях и их аппаратно-программных конфигурациях.
3. Хранение информации о лицензиях на шрифты, условиях использования и сроках действия.
4. Хранение информации об установленных шрифтах на каждой рабочей станции.

Текст чтения диаграммы:

1. Издательство выпускает гарнитуру.
2. Издательство выдаёт лицензию.
3. Гарнитура классифицируется по категории.
4. Гарнитура включает начертание.
5. Начертание содержит алфавит.
6. Алфавит охватывает язык.
7. Начертание участвует в установке.
8. Установка производится на рабочей станции.
9. На рабочую станцию установлена операционная система.
10. Начертание хранится в формате.
11. Гарнитура распространяется по лицензии.

Рис. 1: ER-диаграмма предметной области «Учёт компьютерных шрифтов в электронной типографии»

1.7 Схема объектов

Схема объектов представлена на рисунке 2.

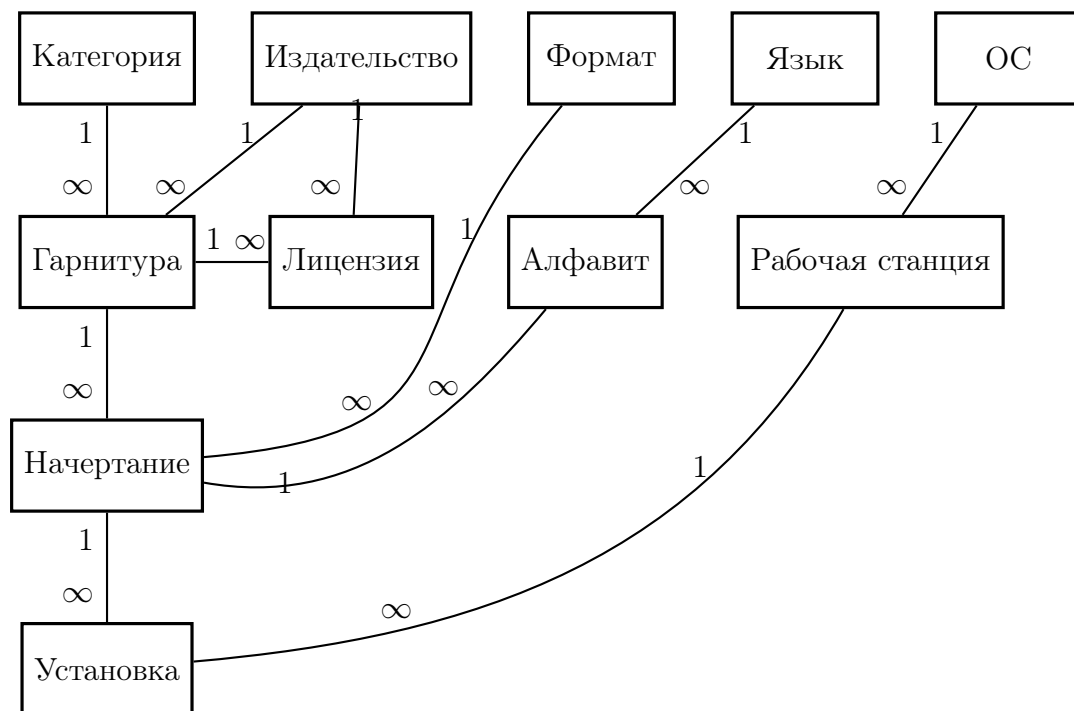


Рис. 2: Схема объектов предметной области «Учёт компьютерных шрифтов в электронной типографии»

2 Проектирование

Схема базы данных приведена на рисунках 3 и 4 на русском и английском языках соответственно.

2.1 Лингвистическое описание схемы базы данных

1. *Издательство* (*id_Издательство* INT, *Название* VARCHAR(100), *Страна* VARCHAR(60), *Сайт* TEXT, *Год_основания* SMALLINT, *Email* VARCHAR(254)).
2. *Гарнитура* (*id_Гарнитура* INT, *Название* VARCHAR(100), *Год_создания* SMALLINT, *Описание* TEXT, *Число_начертаний* SMALLINT, *id_Издательство* INT, *id_Категория* INT).
3. *Категория* (*id_Категория* INT, *Название* VARCHAR(100), *Описание* TEXT, *Иконка* TEXT, *Число_гарнитур* INT).
4. *Лицензия* (*id_Лицензия* INT, *Тип* VARCHAR(30), *Срок_действия* DATE, *Максимальное_число_станций* SMALLINT, *Стоимость* DECIMAL(10,2), *Дата_приобретения* DATE, *id_Издательство* INT, *id_Гарнитура* INT).
5. *Формат* (*id_Формат* INT, *Название* VARCHAR(100), *Расширение* VARCHAR(10), *Год_появления* SMALLINT, *Описание* TEXT, *Поддержка_веба* BOOLEAN).
6. *Начертание* (*id_Начертание* INT, *Название* VARCHAR(100), *Насыщенность* VARCHAR(30), *Наклон* VARCHAR(30), *Размер_файла* INT, *id_Гарнитура* INT, *id_Формат* INT).
7. *Язык* (*id_Язык* INT, *Название* VARCHAR(60), *Код_ISO* CHAR(3), *Направление_письма* VARCHAR(30), *Тип_письменности* VARCHAR(30)).
8. *Алфавит* (*id_Алфавит* INT, *Число_начертаний_в_системе* INT, *Число_поддерживаемых_гарнитур* INT, *Дата_проверки* DATE, *Статус* VARCHAR(30), *id_Начертание* INT, *id_Язык* INT).
9. *Рабочая_станция* (*id_Рабочая_станция* INT, *Инвентарный_номер* VARCHAR(20), *Процессор* VARCHAR(80), *Объём_памяти* INT, *Тип_накопителя* VARCHAR(30), *id_ОС* INT).
10. *ОС* (*id_ОС* INT, *Название* VARCHAR(50), *Версия* VARCHAR(30), *Разрядность* SMALLINT, *Год_выпуска* SMALLINT).
11. *Установка* (*id_Установка* INT, *Дата_установки* DATE, *Дата_удаления* DATE, *Путь_к_файлу* TEXT, *Статус* VARCHAR(30), *Дата_последнего_использования* DATE, *id_Начертание* INT, *id_Рабочая_станция* INT).

2.2 Описание таблиц базы данных

В таблицах 1–11 представлены атрибуты проектируемой базы данных. В полях таблиц перечислены: ключ (*тип ключа «PK» — Primary Key, «FK» — Foreign Key*), названия атрибутов на русском и на английском языках, тип атрибута и ссылка на другую таблицу, если таковая имеется.

Таблица 1: Издательство — Publisher

Издательство — Publisher						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Publisher	id_Издательство	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(100)	NOT NULL	—
3	—	Country	Страна	VARCHAR(60)	NOT NULL	—
4	—	Website	Сайт	TEXT	NULL	—
5	—	Foundation_year	Год_основания	SMALLINT	NOT NULL	—
6	—	Email	Email	VARCHAR(254)	NULL	—

В таблице 1 для URL-адресов выбран тип `TEXT`, так как считаем, что в окне браузера можно ввести URL-адрес любой длины. Тип `VARCHAR(254)` выбран для email-адресов. Их максимальная длина составляет 254 символа согласно RFC 5321¹. `VARCHAR(100)` — для имён гарнитур и начертаний² (аналогично в таблицах 2, 3, 5 и 6) тогда как `VARCHAR(60)` ограничивает названия стран в соответствии с максимальной длиной официальных наименований. Для стоимости лицензии применяется `DECIMAL(10,2)`, обеспечивающий точные денежные вычисления без ошибок округления. Код языка хранится в `CHAR(3)`, так как коды ISO 639³ имеют фиксированную длину в два или три символа.

¹RFC 5321, Section 4.5.3.1 — <https://datatracker.ietf.org/doc/html/rfc5321#section-4.5.3.1>

²Например, самое длинное название начертания, которое удалось найти «Noto Sans Devanagari SemiCondensed ExtraLight Italic», — 52 символа.

³<https://www.iso.org/iso-639-language-code>

Таблица 2: Гарнитура — Font_family

<i>Гарнитура — Font_family</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Font_family	id_Гарнитура	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(100)	NOT NULL	—
3	—	Year_created	Год_создания	SMALLINT	NULL	—
4	—	Description	Описание	TEXT	NULL	—
5	—	Typeface_count	Число_начертаний	SMALLINT	NOT NULL	—
6	FK	id_Publisher	id_Издательство	INT	NOT NULL	Publisher.id_Publisher
7	FK	id_Category	id_Категория	INT	NOT NULL	Category.id_Category

Таблица 3: Категория — Category

<i>Категория — Category</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Category	id_Категория	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(100)	NOT NULL	—
3	—	Description	Описание	TEXT	NULL	—
4	—	Icon	Иконка	TEXT	NULL	—
5	—	Font_family_count	Число_гарнитур	INT	NOT NULL	—

В таблицах 3 и 11 имя директории имеет тип TEXT, т.к. может быть любой длины в рамках операционной системы.

Таблица 4: Лицензия — License

<i>Лицензия — License</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_License	id_Лицензия	INT	NOT NULL	—
2	—	Type	Тип	VARCHAR(30)	NOT NULL	—
3	—	Expiration_date	Срок_действия	DATE	NULL	—
4	—	Max_stations	Макс_число_станций	SMALLINT	NULL	—
5	—	Cost	Стоимость	DECIMAL(10,2)	NOT NULL	—
6	—	Purchase_date	Дата_приобретения	DATE	NOT NULL	—
7	FK	id_Publisher	id_Издательство	INT	NOT NULL	Publisher.id_Publisher
8	FK	id_Font_family	id_Гарнитура	INT	NOT NULL	Font_family.id_Font_family

В таблице 4 тип лицензии хранится в `VARCHAR(30)`, что достаточно для значений вроде «OFL», «Commercial», «Subscription» или «Trial». Поле `Срок_действия` допускает `NULL`, поскольку бессрочные лицензии не имеют даты окончания. Аналогично `Макс_число_станций` допускает `NULL` для лицензий без ограничения на количество установок.

Таблица 5: Формат — Format

<i>Формат — Format</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Format	id_Формат	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(100)	NOT NULL	—
3	—	Extension	Расширение	VARCHAR(10)	NOT NULL	—
4	—	Year_released	Год_появления	SMALLINT	NULL	—
5	—	Description	Описание	TEXT	NULL	—
6	—	Web_support	Поддержка_веба	BOOLEAN	NOT NULL	—

В таблице 5 расширение файла хранится в `VARCHAR(10)`, что покрывает существующие форматы шрифтов: «.ttf», «.otf», «.woff2».

Таблица 6: Начертание — Typeface

<i>Начертание — Typeface</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Typeface	id_Начертание	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(100)	NOT NULL	—
3	—	Weight	Насыщенность	VARCHAR(30)	NOT NULL	—
4	—	Slope	Наклон	VARCHAR(30)	NOT NULL	—
5	—	File_size	Размер_файла	INT	NOT NULL	—
6	FK	id_Font_family	id_Гарнитура	INT	NOT NULL	Font_family.id_Font_family
7	FK	id_Format	id_Формат	INT	NOT NULL	Format.id_Format

В таблице 6 насыщенность и наклон хранятся в `VARCHAR(30)`, что достаточно для стандартных значений: «Thin», «ExtraLight», «Regular», «Bold», «ExtraBold» для насыщенности и «Upright», «Italic», «Oblique» для наклона. Размер файла хранится в `INT` и измеряется в байтах — максимальное значение около 2 ГБ, что с запасом покрывает любые файлы шрифтов.

Таблица 7: Язык — Language

<i>Язык — Language</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Language	id_Язык	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(60)	NOT NULL	—
3	—	ISO_code	Код_ISO	CHAR(3)	NOT NULL	—
4	—	Writing_direction	Направление_письма	VARCHAR(30)	NOT NULL	—
5	—	Script_type	Тип_письменности	VARCHAR(30)	NOT NULL	—

В таблице 7 название языка хранится в `VARCHAR(60)`, что покрывает самые длинные наименования в реестре ISO 639-3⁴, например «Min Nan Chinese» или «Ancient Greek». Направление письма хранится в `VARCHAR(30)` для значений «left-to-right», «right-to-left» или «top-to-bottom». Тип письменности — «alphabetic», «syllabic», «logographic» — также укладывается в `VARCHAR(30)`. Аналогично со статусом в таблице 8 — также хранится в `VARCHAR(30)` и принимает стандартные значения «full», «partial»

⁴ISO 639-3 Code Tables — https://iso639-3.sil.org/code_tables/639/data

или «unsupported». Или в таблице 11 поле Статус принимает значения «active», «disabled» или «removed» — тоже используется тип `VARCHAR(30)`.

Таблица 8: Алфавит — Alphabet

<i>Алфавит — Alphabet</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Alphabet	id_Алфавит	INT	NOT NULL	—
2	—	Typeface_count_in_system	Число_начертаний_в_системе	INT	NOT NULL	—
3	—	Supported_family_count	Число_поддерживаемых_гарнитур	INT	NOT NULL	—
4	—	Verification_date	Дата_проверки	DATE	NULL	—
5	—	Status	Статус	VARCHAR(30)	NOT NULL	—
6	FK	id_Typeface	id_Начертание	INT	NOT NULL	Typeface.id_Typeface
7	FK	id_Language	id_Язык	INT	NOT NULL	Language.id_Language

Таблица 9: Рабочая станция — Workstation

<i>Рабочая станция — Workstation</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Workstation	id_Рабочая_станция	INT	NOT NULL	—
2	—	Inventory_number	Инвентарный_номер	VARCHAR(20)	NOT NULL	—
3	—	Processor	Процессор	VARCHAR(80)	NOT NULL	—
4	—	Memory_size	Объём_памяти	INT	NOT NULL	—
5	—	Storage_type	Тип_накопителя	VARCHAR(30)	NOT NULL	—
6	FK	id_OS	id_ОС	INT	NOT NULL	OS.id_OS

В таблице 9 название процессора хранится в `VARCHAR(80)`: согласно спецификации Intel⁵, название процессора занимает до 48 символов, а запас до 80 учитывает возможные дополнения при ручном вводе. Объём памяти хранится в `INT` и измеряется в мегабайтах. Тип накопителя — `VARCHAR(30)` для значений «SSD», «HDD» или «NVMe SSD». Инвентарный номер — `VARCHAR(20)`, поскольку инвентарные номера обычно содержат буквенно-цифровые коды фиксированной длины, считаем, что в нашей типографии максимальная длина кода — 20 символов.

⁵Intel® Processor Identification and the CPUID Instruction, Application Note 485, Section 3.2.3 — <https://datasheets.chipdb.org/Intel/x86/CPUID/24161831.pdf>

Таблица 10: Операционная система — OS

<i>Операционная система — OS</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_OS	id_ОС	INT	NOT NULL	—
2	—	Name	Название	VARCHAR(50)	NOT NULL	—
3	—	Version	Версия	VARCHAR(30)	NOT NULL	—
4	—	Bitness	Разрядность	SMALLINT	NOT NULL	—
5	—	Release_year	Год_выпуска	SMALLINT	NOT NULL	—

В таблице 10 название операционной системы хранится в `VARCHAR(50)`: согласно реестру IANA⁶, официальное имя ОС может содержать до 40 символов, а запас до 50 учитывает неофициальные наименования. Версия — в `VARCHAR(30)`, поскольку версии могут содержать буквенно-цифровые обозначения, например «24H2», «15.4.1» или «24.04 LTS». Разрядность хранится в `SMALLINT` и принимает значения 32 или 64.

Таблица 11: Установка — Installation

<i>Установка — Installation</i>						
№	Ключ	Название EN	Название RU	Тип	NULL	Ссылка
1	PK	id_Installation	id_Установка	INT	NOT NULL	—
2	—	Installation_date	Дата_установки	DATE	NOT NULL	—
3	—	Removal_date	Дата_удаления	DATE	NULL	—
4	—	File_path	Путь_к_файлу	TEXT	NOT NULL	—
5	—	Status	Статус	VARCHAR(30)	NOT NULL	—
6	—	Last_used_date	Дата_последнего_использования	DATE	NULL	—
7	FK	id_Typeface	id_Начертание	INT	NOT NULL	Typeface.id_Typeface
8	FK	id_Workstation	id_Рабочая_станция	INT	NOT NULL	Workstation.id_Workstation

⁶IANA Operating System Names — <https://www.iana.org/assignments/operating-system-names>

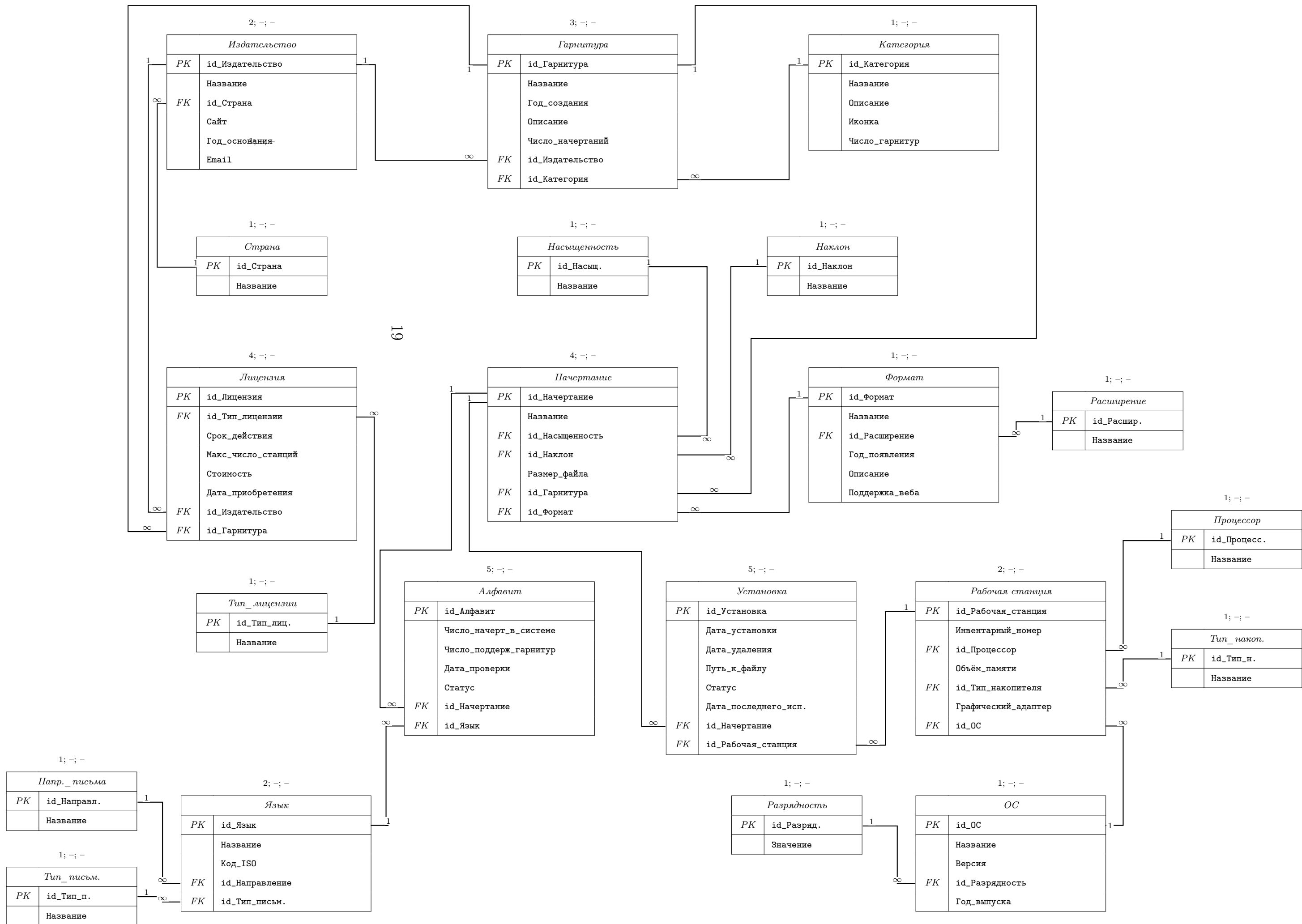
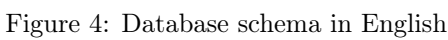


Рис. 3: Схема базы данных на русском языке



2.3 Создание таблиц базы данных и генерация записей

Для создания таблиц базы данных был написан SQL-скрипт для СУБД PostgreSQL (приложение А). В проекте используется база данных `typography`, запущенная в контейнере Docker. Контейнер базы данных создаётся на основе образа PostgreSQL и доступен на локальном порту 5432.

В указанном выше SQL-скрипте последовательно создаются таблицы предметной области: `Category`, `Font_family`, `Format`, `Typeface`, `Language` и `Alphabet`. Порядок создания таблиц выбран с учётом внешних ключей: сначала создаются справочные и независимые таблицы, затем таблицы, которые на них ссылаются.

Структура связей между таблицами, используемыми при программной реализации базы данных, представлена на уменьшенной схеме базы данных на рисунке 5.

Для заполнения базы данных тестовыми данными используется программа на TypeScript (Приложение Б). Подключение к базе данных выполняется при помощи библиотеки `pg`. Для генерации правдоподобных значений используется библиотека `@faker-js/faker`. Запуск скриптов выполняется через `tsx`, что позволяет запускать TypeScript-файлы без отдельной ручной компиляции.

Генерация записей должна выполняться в том же порядке, что и создание зависимостей между таблицами. Сначала заполняются таблицы `Category`, `Format` и `Language`, так как они не зависят от других таблиц. Затем создаются записи в таблице `Font_family`, для каждой из которых выбирается существующая категория. После этого создаются записи в таблице `Typeface`, где для каждого начертания выбираются существующие гарнитура и формат. Последней заполняется таблица `Alphabet`, связывающая существующие языки и начертания.

Количество записей в каждой таблице приведено на схеме базы данных (рис. 5).

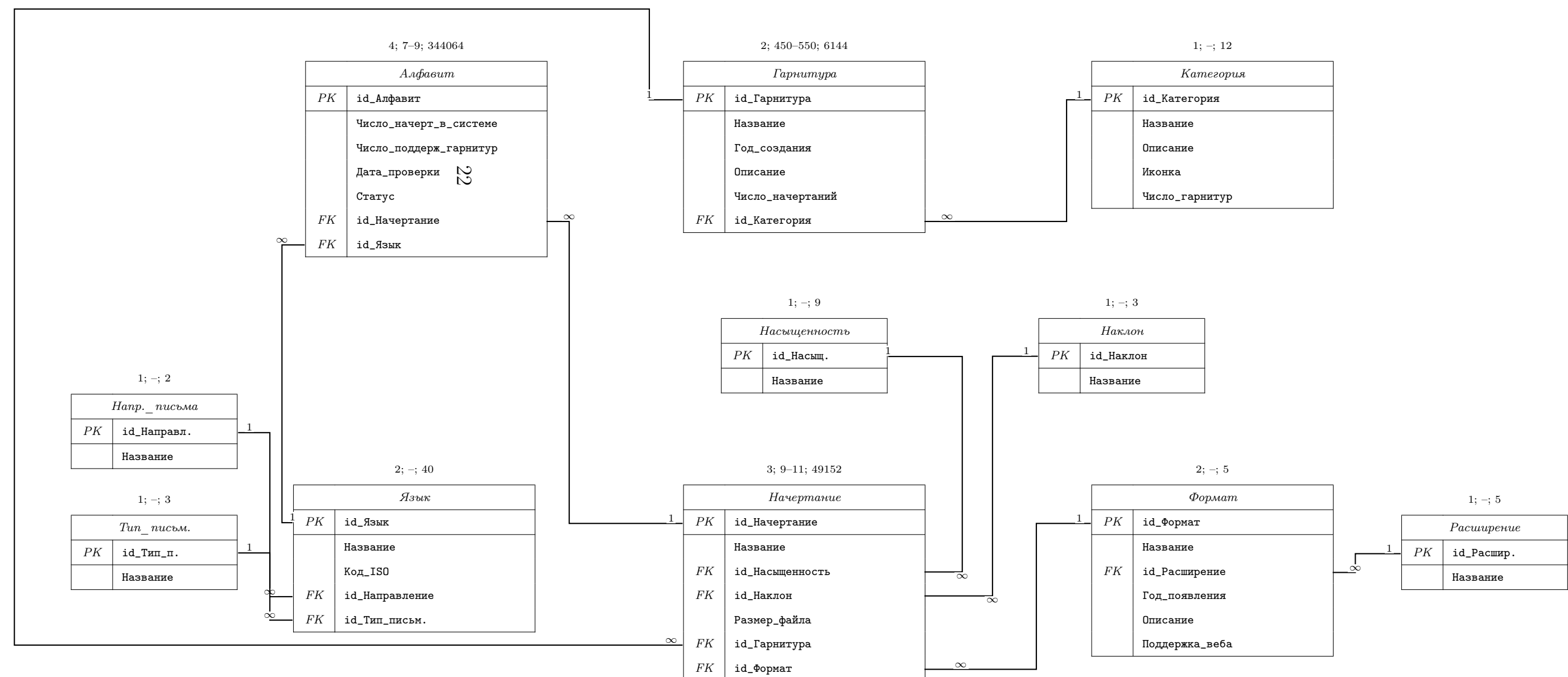


Рис. 3: Программируемая схема базы данных

Приложение А. Исходный код QSL-скрипта для создания таблиц базы данных

```
1 DROP TABLE IF EXISTS Alphabet CASCADE;
2
3 DROP TABLE IF EXISTS Typeface CASCADE;
4
5 DROP TABLE IF EXISTS Font_family CASCADE;
6
7 DROP TABLE IF EXISTS Language CASCADE;
8
9 DROP TABLE IF EXISTS Format CASCADE;
10
11 DROP TABLE IF EXISTS Category CASCADE;
12
13 DROP TABLE IF EXISTS Weight CASCADE;
14
15 DROP TABLE IF EXISTS Slope CASCADE;
16
17 DROP TABLE IF EXISTS Extension CASCADE;
18
19 DROP TABLE IF EXISTS ISO_code CASCADE;
20
21 DROP TABLE IF EXISTS Writing_direction CASCADE;
22
23 DROP TABLE IF EXISTS Script_type CASCADE;
24
25 CREATE TABLE Category (
26     id_Category SERIAL PRIMARY KEY,
27     Name VARCHAR(100) NOT NULL,
28     Description TEXT,
29     Icon TEXT,
30     Font_family_count INT NOT NULL DEFAULT 0
31 );
32
33 CREATE TABLE Weight (
34     id_Weight SERIAL PRIMARY KEY,
35     Name VARCHAR(30) NOT NULL UNIQUE
36 );
37
38 CREATE TABLE Slope (
39     id_Slope SERIAL PRIMARY KEY,
40     Name VARCHAR(30) NOT NULL UNIQUE
41 );
42
43 CREATE TABLE Extension (
44     id_Extension SERIAL PRIMARY KEY,
45     Name VARCHAR(10) NOT NULL UNIQUE
46 );
47
48 CREATE TABLE Writing_direction (
49     id_Writing_direction SERIAL PRIMARY KEY,
50     Name VARCHAR(30) NOT NULL UNIQUE
51 );
52
53 CREATE TABLE Script_type (
54     id_Script_type SERIAL PRIMARY KEY,
55     Name VARCHAR(30) NOT NULL UNIQUE
56 );
57
58 CREATE TABLE Format (
59     id_Format SERIAL PRIMARY KEY,
60     Name VARCHAR(100) NOT NULL,
61     id_Extension INT NOT NULL REFERENCES Extension(id_Extension),
62     Year_released SMALLINT,
63     Description TEXT,
64     Web_support BOOLEAN NOT NULL
65 );
66
67 CREATE TABLE Language (
68     id_Language SERIAL PRIMARY KEY,
69     Name VARCHAR(60) NOT NULL,
70     ISO_code CHAR(3) NOT NULL UNIQUE,
71     id_Writing_direction INT NOT NULL REFERENCES Writing_direction(id_Writing_direction),
72     id_Script_type INT NOT NULL REFERENCES Script_type(id_Script_type)
```

```

73 );
74
75 CREATE TABLE Font_family (
76     id_Font_family SERIAL PRIMARY KEY,
77     Name VARCHAR(100) NOT NULL,
78     Year_created SMALLINT,
79     Description TEXT,
80     Typeface_count SMALLINT NOT NULL DEFAULT 0,
81     id_Category INT NOT NULL REFERENCES Category(id_Category)
82 );
83
84 CREATE TABLE Typeface (
85     id_Typeface SERIAL PRIMARY KEY,
86     Name VARCHAR(100) NOT NULL,
87     id_Weight INT NOT NULL REFERENCES Weight(id_Weight),
88     id_Slope INT NOT NULL REFERENCES Slope(id_Slope),
89     File_size INT NOT NULL,
90     id_Font_family INT NOT NULL REFERENCES Font_family(id_Font_family),
91     id_Format INT NOT NULL REFERENCES Format(id_Format)
92 );
93
94 CREATE TABLE Alphabet (
95     id_Alphabet SERIAL PRIMARY KEY,
96     Typeface_count_in_system INT NOT NULL DEFAULT 0,
97     Supported_family_count INT NOT NULL DEFAULT 0,
98     Verification_date DATE,
99     Status VARCHAR(30) NOT NULL,
100     id_Typeface INT NOT NULL REFERENCES Typeface(id_Typeface),
101     id_Language INT NOT NULL REFERENCES Language(id_Language),
102
103     CONSTRAINT uq_alphabet_typeface_language UNIQUE (id_Typeface, id_Language)
104 );

```

Приложение Б. Программа на TypeScript для заполнения базы данных тестовыми данными

```
1
2 // db.ts
3 import pg from "pg";
4
5 export const pool = new pg.Pool({
6   host: "localhost",
7   port: 5432,
8   database: "typography",
9   user: "ilyasemenov",
10  password: "type",
11 });
12
13
14 // init.ts
15 import { readFileSync } from "fs";
16 import { pool } from "./db.js";
17
18 async function init() {
19   const sql = readFileSync("init.sql", "utf-8");
20   await pool.query(sql);
21   console.log("Tables created successfully");
22   await pool.end();
23 }
24
25 // constants.ts
26 import { faker } from "@faker-js/faker";
27
28 export const BASE_COUNTS = {
29   categories: 12,
30   formats: 5,
31   languages: 40,
32 } as const;
33
34 export const COEFFICIENT_LIMITS = {
35   fontFamiliesPerCategory: { min: 450, max: 550 },
36   typefacesPerFamily: { min: 8, max: 10 },
37   alphabetsPerTypeface: { min: 6, max: 8 },
38 } as const;
39
40 export const COEFFICIENTS = {
41   fontFamiliesPerCategory: faker.number.int(
42     COEFFICIENT_LIMITS.fontFamiliesPerCategory,
43   ),
44
45   typefacesPerFamily: faker.number.int(COEFFICIENT_LIMITS.typefacesPerFamily),
46
47   alphabetsPerTypeface: faker.number.int(
48     COEFFICIENT_LIMITS.alphabetsPerTypeface,
49   ),
50 } as const;
51
52 export const COUNTS = {
53   categories: BASE_COUNTS.categories,
54   formats: BASE_COUNTS.formats,
55   languages: BASE_COUNTS.languages,
56
57   fontFamilies: BASE_COUNTS.categories * COEFFICIENTS.fontFamiliesPerCategory,
58
59   typefaces:
60     BASE_COUNTS.categories *
61     COEFFICIENTS.fontFamiliesPerCategory *
62     COEFFICIENTS.typefacesPerFamily,
63
64   alphabets:
65     BASE_COUNTS.categories *
66     COEFFICIENTS.fontFamiliesPerCategory *
67     COEFFICIENTS.typefacesPerFamily *
68     COEFFICIENTS.alphabetsPerTypeface,
69 } as const;
70 export const categoryNames = [
71   "Serif",
72   "Sans-serif",
```

```

73   "Monospace",
74   "Display",
75   "Handwriting",
76   "Slab-serif",
77   "Geometric",
78   "Humanist",
79   "Grotesque",
80   "Neo-grotesque",
81   "Transitional",
82   "Didone",
83 ];
84
85 export const categoryDescs: Record<string, string> = {
86   Serif: "Typefaces with decorative strokes at the ends of letterforms",
87   "Sans-serif": "Typefaces without decorative strokes",
88   Monospace: "Typefaces where each character occupies the same width",
89   Display: "Typefaces designed for headlines and large sizes",
90   Handwriting: "Typefaces imitating handwritten text",
91   "Slab-serif": "Typefaces with thick rectangular serifs",
92   Geometric: "Sans-serif typefaces based on geometric shapes",
93   Humanist: "Typefaces with organic strokes inspired by calligraphy",
94   Grotesque: "Early sans-serif typefaces with irregular proportions",
95   "Neo-grotesque": "Refined sans-serif typefaces with uniform strokes",
96   Transitional: "Serif typefaces bridging old-style and modern designs",
97   Didone: "High-contrast serif typefaces with thin hairlines",
98 };
99
100 export const formats = [
101   {
102     name: "TrueType",
103     ext: ".ttf",
104     year: 1991,
105     desc: "Scalable font format developed by Apple",
106     web: false,
107   },
108   {
109     name: "OpenType",
110     ext: ".otf",
111     year: 1996,
112     desc: "Scalable font format by Microsoft and Adobe",
113     web: false,
114   },
115   {
116     name: "WOFF",
117     ext: ".woff",
118     year: 2010,
119     desc: "Web Open Font Format",
120     web: true,
121   },
122   {
123     name: "WOFF2",
124     ext: ".woff2",
125     year: 2014,
126     desc: "Compressed Web Open Font Format",
127     web: true,
128   },
129   {
130     name: "TrueType Collection",
131     ext: ".ttc",
132     year: 1994,
133     desc: "Multiple TrueType fonts in a single file",
134     web: false,
135   },
136 ];
137
138 export const languages = [
139   {
140     name: "English",
141     iso: "eng",
142     writingDirection: "left-to-right",
143     scriptType: "alphabetic",
144   },
145   {
146     name: "Russian",
147     iso: "rus",
148     writingDirection: "left-to-right",
149     scriptType: "alphabetic",
150   },

```

```

151 {
152   name: "German",
153   iso: "deu",
154   writingDirection: "left-to-right",
155   scriptType: "alphabetic",
156 },
157 {
158   name: "French",
159   iso: "fra",
160   writingDirection: "left-to-right",
161   scriptType: "alphabetic",
162 },
163 {
164   name: "Spanish",
165   iso: "spa",
166   writingDirection: "left-to-right",
167   scriptType: "alphabetic",
168 },
169 {
170   name: "Italian",
171   iso: "ita",
172   writingDirection: "left-to-right",
173   scriptType: "alphabetic",
174 },
175 {
176   name: "Portuguese",
177   iso: "por",
178   writingDirection: "left-to-right",
179   scriptType: "alphabetic",
180 },
181 {
182   name: "Dutch",
183   iso: "nld",
184   writingDirection: "left-to-right",
185   scriptType: "alphabetic",
186 },
187 {
188   name: "Polish",
189   iso: "pol",
190   writingDirection: "left-to-right",
191   scriptType: "alphabetic",
192 },
193 {
194   name: "Czech",
195   iso: "ces",
196   writingDirection: "left-to-right",
197   scriptType: "alphabetic",
198 },
199 {
200   name: "Slovak",
201   iso: "slk",
202   writingDirection: "left-to-right",
203   scriptType: "alphabetic",
204 },
205 {
206   name: "Ukrainian",
207   iso: "ukr",
208   writingDirection: "left-to-right",
209   scriptType: "alphabetic",
210 },
211 {
212   name: "Belarusian",
213   iso: "bel",
214   writingDirection: "left-to-right",
215   scriptType: "alphabetic",
216 },
217 {
218   name: "Serbian",
219   iso: "srp",
220   writingDirection: "left-to-right",
221   scriptType: "alphabetic",
222 },
223 {
224   name: "Croatian",
225   iso: "hrv",
226   writingDirection: "left-to-right",
227   scriptType: "alphabetic",
228 },

```

```

229 {
230   name: "Bulgarian",
231   iso: "bul",
232   writingDirection: "left-to-right",
233   scriptType: "alphabetic",
234 },
235 {
236   name: "Greek",
237   iso: "ell",
238   writingDirection: "left-to-right",
239   scriptType: "alphabetic",
240 },
241 {
242   name: "Turkish",
243   iso: "tur",
244   writingDirection: "left-to-right",
245   scriptType: "alphabetic",
246 },
247 {
248   name: "Finnish",
249   iso: "fin",
250   writingDirection: "left-to-right",
251   scriptType: "alphabetic",
252 },
253 {
254   name: "Swedish",
255   iso: "swe",
256   writingDirection: "left-to-right",
257   scriptType: "alphabetic",
258 },
259 {
260   name: "Norwegian",
261   iso: "nor",
262   writingDirection: "left-to-right",
263   scriptType: "alphabetic",
264 },
265 {
266   name: "Danish",
267   iso: "dan",
268   writingDirection: "left-to-right",
269   scriptType: "alphabetic",
270 },
271 {
272   name: "Icelandic",
273   iso: "isl",
274   writingDirection: "left-to-right",
275   scriptType: "alphabetic",
276 },
277 {
278   name: "Irish",
279   iso: "gle",
280   writingDirection: "left-to-right",
281   scriptType: "alphabetic",
282 },
283 {
284   name: "Welsh",
285   iso: "cym",
286   writingDirection: "left-to-right",
287   scriptType: "alphabetic",
288 },
289 {
290   name: "Arabic",
291   iso: "ara",
292   writingDirection: "right-to-left",
293   scriptType: "alphabetic",
294 },
295 {
296   name: "Hebrew",
297   iso: "heb",
298   writingDirection: "right-to-left",
299   scriptType: "alphabetic",
300 },
301 {
302   name: "Persian",
303   iso: "fas",
304   writingDirection: "right-to-left",
305   scriptType: "alphabetic",
306 },

```

```

307 {
308   name: "Urdu",
309   iso: "urd",
310   writingDirection: "right-to-left",
311   scriptType: "alphabetic",
312 },
313 {
314   name: "Hindi",
315   iso: "hin",
316   writingDirection: "left-to-right",
317   scriptType: "syllabic",
318 },
319 {
320   name: "Bengali",
321   iso: "ben",
322   writingDirection: "left-to-right",
323   scriptType: "syllabic",
324 },
325 {
326   name: "Tamil",
327   iso: "tam",
328   writingDirection: "left-to-right",
329   scriptType: "syllabic",
330 },
331 {
332   name: "Telugu",
333   iso: "tel",
334   writingDirection: "left-to-right",
335   scriptType: "syllabic",
336 },
337 {
338   name: "Thai",
339   iso: "tha",
340   writingDirection: "left-to-right",
341   scriptType: "syllabic",
342 },
343 {
344   name: "Lao",
345   iso: "lao",
346   writingDirection: "left-to-right",
347   scriptType: "syllabic",
348 },
349 {
350   name: "Chinese",
351   iso: "zho",
352   writingDirection: "left-to-right",
353   scriptType: "logographic",
354 },
355 {
356   name: "Japanese",
357   iso: "jpn",
358   writingDirection: "left-to-right",
359   scriptType: "logographic",
360 },
361 {
362   name: "Korean",
363   iso: "kor",
364   writingDirection: "left-to-right",
365   scriptType: "syllabic",
366 },
367 {
368   name: "Vietnamese",
369   iso: "vie",
370   writingDirection: "left-to-right",
371   scriptType: "alphabetic",
372 },
373 {
374   name: "Indonesian",
375   iso: "ind",
376   writingDirection: "left-to-right",
377   scriptType: "alphabetic",
378 },
379 ];
380
381 export const weights = [
382   "Thin",
383   "ExtraLight",
384   "Light",

```

```

385     "Regular",
386     "Medium",
387     "SemiBold",
388     "Bold",
389     "ExtraBold",
390     "Black",
391 ];
392
393 export const slopes = ["Upright", "Italic", "Oblique"];
394
395 export const alphabetStatuses = ["full", "partial", "unsupported"];
396
397 export const tables = [
398     "Category",
399     "Format",
400     "Language",
401     "Font_family",
402     "Typeface",
403     "Alphabet",
404 ];
405
406 init().catch(console.error);
407
408 // seed.ts
409 import { faker } from "@faker-js/faker";
410 import { pool } from "./db.js";
411 import {
412     BASE_COUNTS,
413     COEFFICIENTS,
414     COUNTS,
415     categoryNames,
416     categoryDescs,
417     formats,
418     languages,
419     weights,
420     slopes,
421     alphabetStatuses,
422     tables,
423 } from "./constants.js";
424 import type { PoolClient } from "pg";
425
426 type IdMap = Map<string, number>;
427
428 type TypefaceRow = {
429     id: number;
430     fontFamilyId: number;
431 };
432
433 class Seeder {
434     private client!: PoolClient;
435
436     private categoryIds: number[] = [];
437     private formatIds: number[] = [];
438     private languageIds: number[] = [];
439     private fontFamilyIds: number[] = [];
440     private typefaces: TypefaceRow[] = [];
441
442     private weightIds: IdMap = new Map();
443     private slopeIds: IdMap = new Map();
444     private extensionIds: IdMap = new Map();
445     private writingDirectionIds: IdMap = new Map();
446     private scriptTypeIds: IdMap = new Map();
447
448     async run() {
449         this.client = await pool.connect();
450
451         try {
452             await this.client.query("BEGIN");
453
454             await this.clearTables();
455
456             await this.seedDictionaries();
457
458             await this.seedCategories();
459             await this.seedFormats();
460             await this.seedLanguages();
461
462             await this.seedFontFamilies();

```

```

463     await this.seedTypefaces();
464     await this.seedAlphabets();
465
466     await this.updateCategoryCounts();
467
468     await this.client.query("COMMIT");
469
470     console.log("\nSeeding complete!");
471     await this.printSummary();
472   } catch (error) {
473     await this.client.query("ROLLBACK");
474     throw error;
475   } finally {
476     this.client.release();
477     await pool.end();
478   }
479 }
480
481 private async clearTables() {
482   await this.client.query(`
483     TRUNCATE TABLE
484       Alphabet,
485       Typeface,
486       Font_family,
487       Language,
488       Format,
489       Category,
490       Weight,
491       Slope,
492       Extension,
493       Writing_direction,
494       Script_type
495     RESTART IDENTITY CASCADE
496   `);
497
498   console.log("Tables cleared");
499 }
500
501 private async seedDictionaries() {
502   await this.seedWeightDictionary();
503   await this.seedSlopeDictionary();
504   await this.seedExtensionDictionary();
505   await this.seedWritingDirectionDictionary();
506   await this.seedScriptTypeDictionary();
507 }
508
509 private async seedWeightDictionary() {
510   for (const name of weights) {
511     const result = await this.client.query(
512       `
513         INSERT INTO Weight (Name)
514         VALUES ($1)
515         RETURNING id_Weight
516       `,
517       [name],
518     );
519
520     this.weightIds.set(name, result.rows[0].id_weight);
521   }
522
523   console.log(`Weight: ${this.weightIds.size}`);
524 }
525
526 private async seedSlopeDictionary() {
527   for (const name of slopes) {
528     const result = await this.client.query(
529       `
530         INSERT INTO Slope (Name)
531         VALUES ($1)
532         RETURNING id_Slope
533       `,
534       [name],
535     );
536
537     this.slopeIds.set(name, result.rows[0].id_slope);
538   }
539
540   console.log(`Slope: ${this.slopeIds.size}`);

```

```

541 }
542
543 private async seedExtensionDictionary() {
544     const uniqueExtensions = [...new Set(formats.map((format) => format.ext))];
545
546     for (const extension of uniqueExtensions) {
547         const result = await this.client.query(
548             `
549             INSERT INTO Extension (Name)
550             VALUES ($1)
551             RETURNING id_Extension
552             `,
553             [extension],
554         );
555
556         this.extensionIds.set(extension, result.rows[0].id_extension);
557     }
558
559     console.log(`Extension: ${this.extensionIds.size}`);
560 }
561
562 private async seedWritingDirectionDictionary() {
563     const selectedLanguages = languages.slice(0, BASE_COUNTS.languages);
564     const uniqueWritingDirections = [
565         ...new Set(
566             selectedLanguages.map((language) => language.writingDirection),
567         ),
568     ];
569
570     for (const name of uniqueWritingDirections) {
571         const result = await this.client.query(
572             `
573             INSERT INTO Writing_direction (Name)
574             VALUES ($1)
575             RETURNING id_Writing_direction
576             `,
577             [name],
578         );
579
580         this.writingDirectionIds.set(name, result.rows[0].id_writing_direction);
581     }
582
583     console.log(`Writing_direction: ${this.writingDirectionIds.size}`);
584 }
585
586 private async seedScriptTypeDictionary() {
587     const selectedLanguages = languages.slice(0, BASE_COUNTS.languages);
588     const uniqueScriptTypes = [
589         ...new Set(selectedLanguages.map((language) => language.scriptType)),
590     ];
591
592     for (const name of uniqueScriptTypes) {
593         const result = await this.client.query(
594             `
595             INSERT INTO Script_type (Name)
596             VALUES ($1)
597             RETURNING id_Script_type
598             `,
599             [name],
600         );
601
602         this.scriptTypeIds.set(name, result.rows[0].id_script_type);
603     }
604
605     console.log(`Script_type: ${this.scriptTypeIds.size}`);
606 }
607
608 private async seedCategories() {
609     const selectedCategories = categoryNames.slice(0, BASE_COUNTS.categories);
610
611     for (const name of selectedCategories) {
612         const result = await this.client.query(
613             `
614             INSERT INTO Category
615             (Name, Description, Icon, Font_family_count)
616             VALUES
617             ($1, $2, $3, 0)
618             RETURNING id_Category

```

```

619         `
620         [name, categoryDescs[name], `/icons/${this.slugify(name)}.svg`],
621     );
622
623     this.categoryIds.push(result.rows[0].id_category);
624 }
625
626 console.log(`Category: ${this.categoryIds.length}`);
627 }
628
629 private async seedFormats() {
630     const selectedFormats = formats.slice(0, BASE_COUNTS.formats);
631
632     for (const format of selectedFormats) {
633         const extensionId = this.getRequiredId(
634             this.extensionIds,
635             format.ext,
636             "Extension",
637         );
638
639         const result = await this.client.query(
640             `
641             INSERT INTO Format
642             (Name, id_Extension, Year_released, Description, Web_support)
643             VALUES
644             ($1, $2, $3, $4, $5)
645             RETURNING id_Format
646             `,
647             [format.name, extensionId, format.year, format.desc, format.web],
648         );
649
650         this.formatIds.push(result.rows[0].id_format);
651     }
652
653     console.log(`Format: ${this.formatIds.length}`);
654 }
655
656 private async seedLanguages() {
657     const selectedLanguages = languages.slice(0, BASE_COUNTS.languages);
658
659     for (const language of selectedLanguages) {
660         const writingDirectionId = this.getRequiredId(
661             this.writingDirectionIds,
662             language.writingDirection,
663             "Writing_direction",
664         );
665
666         const scriptTypeId = this.getRequiredId(
667             this.scriptTypeIds,
668             language.scriptType,
669             "Script_type",
670         );
671
672         const result = await this.client.query(
673             `
674             INSERT INTO Language
675             (
676                 Name,
677                 ISO_code,
678                 id_Writing_direction,
679                 id_Script_type
680             )
681             VALUES
682             ($1, $2, $3, $4)
683             RETURNING id_Language
684             `,
685             [language.name, language.iso, writingDirectionId, scriptTypeId],
686         );
687
688         this.languageIds.push(result.rows[0].id_language);
689     }
690
691     console.log(`Language: ${this.languageIds.length}`);
692 }
693
694 private async seedFontFamilies() {
695     let created = 0;
696

```

```

697     const batchSize = 1000;
698     let values: unknown[] = [];
699     let placeholders: string[] = [];
700
701     for (const categoryId of this.categoryIds) {
702         for (let i = 0; i < COEFFICIENTS.fontFamiliesPerCategory; i++) {
703             const index = values.length;
704
705             placeholders.push(
706                 `(${index + 1}, ${index + 2}, ${index + 3}, ${index + 4}, ${index + 5})`,
707             );
708
709             values.push(
710                 this.makeFontFamilyName(created),
711                 faker.number.int({ min: 1950, max: 2024 }),
712                 faker.lorem.sentence(),
713                 COEFFICIENTS.typefacesPerFamily,
714                 categoryId,
715             );
716
717             created++;
718
719             if (placeholders.length >= batchSize) {
720                 await this.insertFontFamilyBatch(placeholders, values);
721                 values = [];
722                 placeholders = [];
723             }
724         }
725
726         console.log(
727             `Font_family by category: ${this.categoryIds.indexOf(categoryId) +
              ↳ 1}/${this.categoryIds.length}`,
728         );
729     }
730
731     if (placeholders.length > 0) {
732         await this.insertFontFamilyBatch(placeholders, values);
733     }
734
735     console.log(`Font_family: ${this.fontFamilyIds.length}`);
736 }
737
738 private async insertFontFamilyBatch(
739     placeholders: string[],
740     values: unknown[],
741 ) {
742     const result = await this.client.query(
743         `
744             INSERT INTO Font_family
745             (
746                 Name,
747                 Year_created,
748                 Description,
749                 Typeface_count,
750                 id_Category
751             )
752             VALUES
753                 ${placeholders.join(", ")}
754             RETURNING id_Font_family
755         `,
756         values,
757     );
758
759     for (const row of result.rows) {
760         this.fontFamilyIds.push(row.id_font_family);
761     }
762 }
763
764 private async seedTypefaces() {
765     let created = 0;
766
767     const batchSize = 1000;
768     let values: unknown[] = [];
769     let placeholders: string[] = [];
770
771     for (const fontFamilyId of this.fontFamilyIds) {
772         const familyName = this.makeTypefaceFamilyPrefix(fontFamilyId);

```

```

773
774 for (let i = 0; i < COEFFICIENTS.typefacesPerFamily; i++) {
775     const weight = weights[i % weights.length];
776     const slope = slopes[i % slopes.length];
777
778     const weightId = this.getRequiredId(this.weightIds, weight, "Weight");
779     const slopeId = this.getRequiredId(this.slopeIds, slope, "Slope");
780
781     const formatId = this.formatIds[i % this.formatIds.length];
782
783     const typefaceName =
784         slope === "Upright"
785             ? `${familyName} ${weight}`
786             : `${familyName} ${weight} ${slope}`;
787
788     const index = values.length;
789
790     placeholders.push(
791         `($${index + 1}, $${index + 2}, $${index + 3}, $${index + 4}, $${index + 5}, $${index + 6})`,
792     );
793
794     values.push(
795         typefaceName.slice(0, 100),
796         weightId,
797         slopeId,
798         faker.number.int({ min: 30_000, max: 400_000 }),
799         fontFamilyId,
800         formatId,
801     );
802
803     created++;
804
805     if (placeholders.length >= batchSize) {
806         await this.insertTypefaceBatch(placeholders, values);
807         values = [];
808         placeholders = [];
809
810         if (created % 5000 === 0) {
811             console.log(`Typeface: ${created}/${COUNTS.typefaces}`);
812         }
813     }
814 }
815
816 if (placeholders.length > 0) {
817     await this.insertTypefaceBatch(placeholders, values);
818 }
819
820 console.log(`Typeface: ${this.typefaces.length}`);
821 }
822
823 private async insertTypefaceBatch(placeholders: string[], values: unknown[]) {
824     const result = await this.client.query(
825         `
826         INSERT INTO Typeface
827         (
828             Name,
829             id_Weight,
830             id_Slope,
831             File_size,
832             id_Font_family,
833             id_Format
834         )
835         VALUES
836         ${placeholders.join(", ")}
837         RETURNING id_Typeface, id_Font_family
838     `,
839         values,
840     );
841
842     for (const row of result.rows) {
843         this.typefaces.push({
844             id: row.id_typeface,
845             fontFamilyId: row.id_font_family,
846         });
847     }
848 }
849

```

```

850
851 private async seedAlphabets() {
852     let created = 0;
853
854     const batchSize = 5000;
855     let values: unknown[] = [];
856     let placeholders: string[] = [];
857
858     for (const typeface of this.typefaces) {
859         const selectedLanguageIds = this.pickLanguagesForTypeface(
860             typeface.id,
861             COEFFICIENTS.alphabetsPerTypeface,
862         );
863
864         for (const languageId of selectedLanguageIds) {
865             const index = values.length;
866
867             placeholders.push(
868                 `(${placeholderIndex + 1}, ${placeholderIndex + 2}, ${placeholderIndex + 3}, ${placeholderIndex + 4}, ${placeholderIndex + 5}, ${placeholderIndex + 6})`,
869             );
870
871             values.push(
872                 this.makeTypefaceCountInSystem(),
873                 this.makeSupportedFamilyCount(),
874                 this.makeVerificationDate(),
875                 this.makeAlphabetStatus(),
876                 typeface.id,
877                 languageId,
878             );
879
880             created++;
881
882             if (placeholders.length >= batchSize) {
883                 await this.insertAlphabetBatch(placeholders, values);
884                 values = [];
885                 placeholders = [];
886
887                 if (created % 50_000 === 0) {
888                     console.log(`Alphabet: ${created}/${COUNTS.alphabets}`);
889                 }
890             }
891         }
892     }
893
894     if (placeholders.length > 0) {
895         await this.insertAlphabetBatch(placeholders, values);
896     }
897
898     console.log(`Alphabet: ${created}`);
899 }
900
901 private async insertAlphabetBatch(placeholders: string[], values: unknown[]) {
902     await this.client.query(
903         `
904         INSERT INTO Alphabet
905         (
906             Typeface_count_in_system,
907             Supported_family_count,
908             Verification_date,
909             Status,
910             id_Typeface,
911             id_Language
912         )
913         VALUES
914             ${placeholders.join(", ")}
915         `,
916         values,
917     );
918 }
919
920 private async updateCategoryCounts() {
921     await this.client.query(
922         `
923         UPDATE Category
924         SET Font_family_count = (
925             SELECT COUNT(*)
926             FROM Font_family
927             WHERE Font_family.id_Category = Category.id_Category
928         )
929     `
930     );
931 }

```

```

928     `);
929
930     console.log("Category counts updated");
931 }
932
933 private pickLanguagesForTypeface(
934     typefaceId: number,
935     count: number,
936 ): number[] {
937     const result: number[] = [];
938
939     const start = typefaceId % this.languageIds.length;
940
941     for (let i = 0; i < count; i++) {
942         const languageId =
943             this.languageIds[(start + i) % this.languageIds.length];
944         result.push(languageId);
945     }
946
947     return result;
948 }
949
950 private makeFontFamilyName(index: number): string {
951     const adjective = faker.word.adjective();
952     const noun = faker.word.noun();
953
954     const generated = `${adjective} ${noun}`
955         .split(" ")
956         .map((part) => part.charAt(0).toUpperCase() + part.slice(1))
957         .join(" ");
958
959     return `${generated} ${index + 1}`.slice(0, 100);
960 }
961
962 private makeTypefaceFamilyPrefix(fontFamilyId: number): string {
963     return `Family ${fontFamilyId}`;
964 }
965
966 private makeTypefaceCountInSystem(): number {
967     return faker.number.int({
968         min: 1,
969         max: COEFFICIENTS.typefacesPerFamily,
970     });
971 }
972
973 private makeSupportedFamilyCount(): number {
974     return faker.number.int({
975         min: 1,
976         max: COEFFICIENTS.fontFamiliesPerCategory,
977     });
978 }
979
980 private makeVerificationDate(): string | null {
981     return (
982         faker.helpers.maybe(
983             () => faker.date.past({ years: 2 }).toISOString().split("T")[0],
984             { probability: 0.7 },
985         ) ?? null
986     );
987 }
988
989 private makeAlphabetStatus(): string {
990     return faker.helpers.arrayElement(alphabetStatuses);
991 }
992
993 private getRequiredId(map: IdMap, key: string, tableName: string): number {
994     const id = map.get(key);
995
996     if (id === undefined) {
997         throw new Error(`Missing dictionary value in ${tableName}: ${key}`);
998     }
999
1000     return id;
1001 }
1002
1003 private slugify(value: string): string {
1004     return value.toLowerCase().replaceAll(" ", "-").replaceAll("_", "-");
1005 }

```

```

1006
1007 private async printSummary() {
1008     let total = 0;
1009
1010     console.log("\nTable counts:");
1011
1012     for (const table of tables) {
1013         const result = await pool.query(`SELECT COUNT(*) FROM ${table}`);
1014         const count = Number(result.rows[0].count);
1015
1016         total += count;
1017         console.log(`  ${table}: ${count}`);
1018     }
1019
1020     console.log(`  TOTAL: ${total}`);
1021 }
1022 }
1023
1024 new Seeder().run().catch((error) => {
1025     console.error(error);
1026     process.exit(1);
1027 });
1028
1029

```